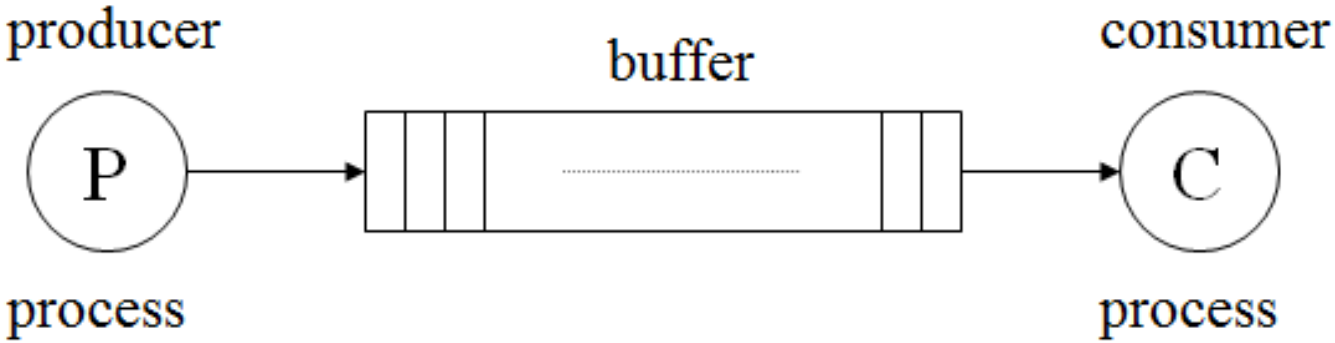
Systems programming

Winter 2024-25

Producer-consumer problem

Producer/Consumer (bounded buffer) Problem



- Assume shared, finite size buffer
- from time to time
  - the producer adds items to buffer
  - the consumer removes items from buffer
- buffer is shared resource $\rightarrow$ synchronized access is required
  - the consumer must wait when the buffer is empty
  - the producer must wait when the buffer is full

Bounded buffer example

- A bounded buffer is used when you pipe the output of one program into another
- Example: grep foo file.txt | wc -l
  - the grep process is the producer
  - The wc process is the consumer
  - Between them is an in-kernel bounded buffer

Producer-Consumer Model

- Shared data

```
sem_t full, empty;
```

- Initially

```
full = 0;          /* The number of full buffers */
empty = MAX;       /* The number of empty buffers */
```

First Attempt: MAX = 1

```
1   sem_t empty;
2   sem_t full;
3
4   void *producer(void *arg) {
5       int i;
6       for (i = 0; i < loops; i++) {
7           sem_wait(&empty);           // line P1
8           put(i);                     // line P2
9           sem_post(&full);            // line P3
10      }
11  }
12
13  void *consumer(void *arg) {
14      int i, tmp = 0;
15      while (tmp != -1) {
16          sem_wait(&full);            // line C1
17          tmp = get();                // line C2
18          sem_post(&empty);           // line C3
19          printf("%d\n", tmp);
20      }
21  }
22
23  int main(int argc, char *argv[]) {
24      // ...
25      sem_init(&empty, 0, MAX); // MAX buffers are empty to begin with...
26      sem_init(&full, 0, 0);    // ... and 0 are full
27      // ...
28  }
```

```
1   int buffer[MAX];
2   int fill = 0;
3   int use  = 0;
4
5   void put(int value) {
6       buffer[fill] = value;
7       fill = (fill + 1) % MAX;
8   }
9
10  int get() {
11      int tmp = buffer[use];
12      use = (use + 1) % MAX;
13      return tmp;
14  }
```

Put and Get routines

First Attempt: MAX = 10?

```
1   sem_t empty;
2   sem_t full;
3
4   void *producer(void *arg) {
5       int i;
6       for (i = 0; i < loops; i++) {
7           sem_wait(&empty);           // line P1
8           put(i);                     // line P2
9           sem_post(&full);            // line P3
10      }
11  }
12
13  void *consumer(void *arg) {
14      int i, tmp = 0;
15      while (tmp != -1) {
16          sem_wait(&full);            // line C1
17          tmp = get();                // line C2
18          sem_post(&empty);           // line C3
19          printf("%d\n", tmp);
20      }
21  }
22
23  int main(int argc, char *argv[]) {
24      // ...
25      sem_init(&empty, 0, MAX); // MAX buffers are empty to begin with...
26      sem_init(&full, 0, 0);    // ... and 0 are full
27      // ...
28  }
```

```
1   int buffer[MAX];
2   int fill = 0;
3   int use  = 0;
4
5   void put(int value) {
6       buffer[fill] = value;
7       fill = (fill + 1) % MAX;
8   }
9
10  int get() {
11      int tmp = buffer[use];
12      use = (use + 1) % MAX;
13      return tmp;
14  }
```

Put and Get routines

First Attempt: MAX = 10?

```
fill = 0
empty = 10
```

Producer 0: **Running**

```
void *producer(void *arg) {
    int i;
    for (i = 0; i < loops; i++) {
        sem_wait(&empty);
        put(i);
        sem_post(&full);
    }
}
```

Producer 1: Runnable

```
void *producer(void *arg) {
    int i;
    for (i = 0; i < loops; i++) {
        sem_wait(&empty);
        put(i);
        sem_post(&full);
    }
}
```

First Attempt: MAX = 10?

```
fill = 0
empty = 9
```

Producer 0: **Running**

```
void *producer(void *arg) {
    int i;
    for (i = 0; i < loops; i++) {
        sem_wait(&empty);
        put(i);
        sem_post(&full);
    }
}
```

Producer 1: Runnable

```
void *producer(void *arg) {
    int i;
    for (i = 0; i < loops; i++) {
        sem_wait(&empty);
        put(i);
        sem_post(&full);
    }
}
```

```
void put(int value) {
    buffer[fill] = value;
    fill = (fill + 1) % MAX;
}
```

First Attempt: MAX = 10?

```
fill = 0
empty = 9
```

Producer 0: **Running**

```
void *producer(void *arg) {
    int i;
    for (i = 0; i < loops; i++) {
        sem_wait(&empty);
        put(i);
        sem_post(&full);
    }
}
```

Producer 1: Runnable

```
void *producer(void *arg) {
    int i;
    for (i = 0; i < loops; i++) {
        sem_wait(&empty);
        put(i);
        sem_post(&full);
    }
}
```

```
void put(int value) {
    buffer[fill] = value;
    Interrupted ...
    fill = (fill + 1) % MAX;
}
```

First Attempt: MAX = 10?

```
fill = 0
empty = 9
```

Producer 0: **Sleeping**

```
void *producer(void *arg) {
    int i;
    for (i = 0; i < loops; i++) {
        sem_wait(&empty);
        put(i);
        sem_post(&full);
    }
}
```

Producer 1: Runnable

```
void *producer(void *arg) {
    int i;
    for (i = 0; i < loops; i++) {
        sem_wait(&empty);
        put(i);
        sem_post(&full);
    }
}
```

```
void put(int value) {
    buffer[fill] = value;
    Interrupted ...
    fill = (fill + 1) % MAX;
}
```

First Attempt: MAX = 10?

```
fill = 0
empty = 9
```

Producer 0: Runnable

```
void *producer(void *arg) {
    int i;
    for (i = 0; i < loops; i++) {
        sem_wait(&empty);
        put(i);
        sem_post(&full);
    }
}
```

Producer 1: **Running**

```
void *producer(void *arg) {
    int i;
    for (i = 0; i < loops; i++) {
        sem_wait(&empty);
        put(i);
        sem_post(&full);
    }
}
```

```
void put(int value) {
    buffer[fill] = value;
    Interrupted ...
    fill = (fill + 1) % MAX;
}
```

First Attempt: MAX = 10?

```
fill = 0
Overwrite!
empty = 8
```

Producer 0: Runnable

```
void *producer(void *arg) {
    int i;
    for (i = 0; i < loops; i++) {
        sem_wait(&empty);
        put(i);
        sem_post(&full);
    }
}
```

Producer 1: **Running**

```
void *producer(void *arg) {
    int i;
    for (i = 0; i < loops; i++) {
        sem_wait(&empty);
        put(i);
        sem_post(&full);
    }
}
```

```
void put(int value) {
    buffer[fill] = value;
    Interrupted ...
    fill = (fill + 1) % MAX;
}
```

```
void put(int value) {
    buffer[fill] = value;
    fill = (fill + 1) % MAX;
}
```

Producer-Consumer Model

- Shared data

```
sem_t full, empty, mutex;
```

- Initially

```
full = 0;          /* The number of full buffers */
empty = MAX;       /* The number of empty buffers */
mutex = 1;         /* Semaphore controlling the access
                      to the buffer pool */
```

Add Mutual Exclusion

```
1   sem_t empty;
2   sem_t full;
3   sem_t mutex;
4
5   void *producer(void *arg) {
6       int i;
7       for (i = 0; i < loops; i++) {
8           sem_wait(&mutex);           // line p0 (NEW LINE)
9           sem_wait(&empty);           // line p1
10          put(i);                     // line p2
11          sem_post(&full);            // line p3
12          sem_post(&mutex);           // line p4 (NEW LINE)
13      }
14  }
15
16  void *consumer(void *arg) {
17      int i;
18      for (i = 0; i < loops; i++) {
19          sem_wait(&mutex);           // line c0 (NEW LINE)
20          sem_wait(&full);            // line c1
21          int tmp = get();            // line c2
22          sem_post(&empty);           // line c3
23          sem_post(&mutex);           // line c4 (NEW LINE)
24          printf("%d\n", tmp);
25      }
26  }
27
28  int main(int argc, char *argv[]) {
29      // ...
30      sem_init(&empty, 0, MAX); // MAX buffers are empty to begin with...
31      sem_init(&full, 0, 0);    // ... and 0 are full
32      sem_init(&mutex, 0, 1);   // mutex=1 because it is a lock (NEW LINE)
33      // ...
34  }
```

Add Mutual Exclusion

```
1   sem_t empty;
2   sem_t full;
3   sem_t mutex;
4
5   void *producer(void *arg) {
6       int i;
7       for (i = 0; i < loops; i++) {
8           sem_wait(&mutex);           // line p0 (NEW LINE)
9           sem_wait(&empty);           // line p1
10          put(i);                     // line p2
11          sem_post(&full);            // line p3
12          sem_post(&mutex);           // line p4 (NEW LINE)
13      }
14  }
15
16  void *consumer(void *arg) {
17      int i;
18      for (i = 0; i < loops; i++) {
19          sem_wait(&mutex);           // line c0 (NEW LINE)
20          sem_wait(&full);            // line c1
21          int tmp = get();            // line c2
22          sem_post(&empty);           // line c3
23          sem_post(&mutex);           // line c4 (NEW LINE)
24          printf("%d\n", tmp);
25      }
26  }
27
28  int main(int argc, char *argv[]) {
29      // ...
30      sem_init(&empty, 0, MAX); // MAX buffers are empty to begin with...
31      sem_init(&full, 0, 0);    // ... and 0 are full
32      sem_init(&mutex, 0, 1);   // mutex=1 because it is a lock (NEW LINE)
33      // ...
34  }
```

What if consumer gets to run first??

Add Mutual Exclusion

```
mutex = 1
full = 0
empty = 10
```

Producer 0: Runnable

```
void *producer(void *arg) {
    int i;
    for (i = 0; i < loops; i++) {
        sem_wait(&mutex);
        sem_wait(&empty);
        put(i);
        sem_post(&full);
        sem_post(&mutex);
    }
}
```

Consumer 0: **Running**

```
void *consumer(void *arg) {
    int i;
    for (i = 0; i < loops; i++) {
        sem_wait(&mutex);
        sem_wait(&full);
        int tmp = get();
        sem_post(&empty);
        sem_post(&mutex);
        printf("%d\n", tmp);
    }
}
```

Add Mutual Exclusion

```
mutex = 0
full = 0
empty = 10
```

Producer 0: Runnable

```
void *producer(void *arg) {
    int i;
    for (i = 0; i < loops; i++) {
        sem_wait(&mutex);
        sem_wait(&empty);
        put(i);
        sem_post(&full);
        sem_post(&mutex);
    }
}
```

Consumer 0: Runnable

```
void *consumer(void *arg) {
    int i;
    for (i = 0; i < loops; i++) {
        sem_wait(&mutex);
        sem_wait(&full);
        int tmp = get();
        sem_post(&empty);
        sem_post(&mutex);
        printf("%d\n", tmp);
    }
}
```

Add Mutual Exclusion

```
mutex = 0
full = 0
empty = 10
```

Producer 0: **Running**

```
void *producer(void *arg) {
    int i;
    for (i = 0; i < loops; i++) {
        sem_wait(&mutex);
        sem_wait(&empty);
        put(i);
        sem_post(&full);
        sem_post(&mutex);
    }
}
```

Consumer 0: Runnable

```
void *consumer(void *arg) {
    int i;
    for (i = 0; i < loops; i++) {
        sem_wait(&mutex);
        sem_wait(&full);
        int tmp = get();
        sem_post(&empty);
        sem_post(&mutex);
        printf("%d\n", tmp);
    }
}
```

Add Mutual Exclusion

Deadlock!!

mutex = 0

full = 0

empty = 10

Producer 0: Running

```
void *producer(void *arg) {
    int i;
    for (i = 0; i < loops; i++) {
        sem_wait(&mutex);
        sem_wait(&empty);
        put(i);
        sem_post(&full);
        sem_post(&mutex);
    }
}
```

Consumer 0: Runnable

```
void *consumer(void *arg) {
    int i;
    for (i = 0; i < loops; i++) {
        sem_wait(&mutex);
        sem_wait(&full);
        int tmp = get();
        sem_post(&empty);
        sem_post(&mutex);
        printf("%d\n", tmp);
    }
}
```

Correct Mutual Exclusion

```
1   sem_t empty;
2   sem_t full;
3   sem_t mutex;
4
5   void *producer(void *arg) {
6       int i;
7       for (i = 0; i < loops; i++) {
8           sem_wait(&empty);           // line p1
9           sem_wait(&mutex);           // line p1.5 (MOVED MUTEX HERE...)
10          put(i);                     // line p2
11          sem_post(&mutex);           // line p2.5 (... AND HERE)
12          sem_post(&full);            // line p3
13      }
14  }
15
16  void *consumer(void *arg) {
17      int i;
18      for (i = 0; i < loops; i++) {
19          sem_wait(&full);            // line c1
20          sem_wait(&mutex);           // line c1.5 (MOVED MUTEX HERE...)
21          int tmp = get();            // line c2
22          sem_post(&mutex);           // line c2.5 (... AND HERE)
23          sem_post(&empty);           // line c3
24          printf("%d\n", tmp);
25      }
26  }
27
28  int main(int argc, char *argv[]) {
29      // ...
30      sem_init(&empty, 0, MAX); // MAX buffers are empty to begin with...
31      sem_init(&full, 0, 0);    // ... and 0 are full
32      sem_init(&mutex, 0, 1);   // mutex=1 because it is a lock
33      // ...
34  }
```

Mutex wraps just around critical section!

Mutex wraps just around critical section!

137

One more example

```
typedef struct account {
  int balance = 0;
  Mutex account_mutex;
} account_type;

void withdraw(account_type *acc, int amount) {...}
void synch_withdraw(account_type *acc, int amount) {
  lock(acc->account_mutex);
  withdraw(acc, amount);
  unlock(acc->account_mutex);
}

void deposit(account_type *acc, int amount) { ... }
void synch_deposit(account_type *acc, int amount) {
  lock(acc->account_mutex);
  deposit(acc, amount);
  unlock(acc->account_mutex);
}
...
```

One more example

```
// Client code
void move(account_type *acc1,
          account_type *acc2, int amount) {
  synch_withdraw(acc1, amount);
  synch_deposit(acc2, amount);
}
```

- Problem: atomicity violation
  - another thread can observe the modification of account balances half-complete

One more example

```
// Client code
void move(account_type *acc1,
          account_type *acc2, int amount) {
  synch_withdraw(acc1, amount);
  synch_deposit(acc2, amount);
}
```

- Problem: atomicity violation
  - another thread can observe the modification of account balances half-complete
  - how can we prevent this?

One more try

```
// Client
void atomic_move(account_type *acc1,
          account_type *acc2, int amount) {
  lock(acc1->account_mutex);
  lock(acc2->account_mutex);
  withdraw(acc1, amount);
  deposit(acc2, amount);
  unlock(acc2->account_mutex);
  unlock(acc1->account_mutex);
}
```

- Deadlock possible?

One more try

```
// Client
void atomic_move(account_type *acc1,
          account_type *acc2, int amount) {
  lock(acc1->account_mutex);
  lock(acc2->account_mutex);
  withdraw(acc1, amount);
  deposit(acc2, amount);
  unlock(acc2->account_mutex);
  unlock(acc1->account_mutex);
}
```

- Deadlock possible?
  - move(s,t,...): concurrent with move(t,s,...)
  - move(s,s,...): self-deadlock